Recap: Repetition Structures

- while structure: loop while the condition is True.

Syntax:

```
while condition:
    statement
    statement
```

while-block

True

False = Out of the loop

Ex

```
n=0
while n<=5:
    print(n)
    n+=2
print("Done")
```

Short-hand n=n+2

Outside of the while

Output:
0
2
4
Done

#1
n<=5 → 0<=5
print(0)
n+=2 = 0+2 = $\boxed{2}$

#2
n<=5 → 2<=5
print(2)
n+=2 = 2+2 = $\boxed{4}$

#3
n<=5 → 4<=5
print(4)
n+=2 = 4+2 = $\boxed{6}$

#4
n<=5 → 6<=5 → False → Out!

Q19: Find the sum from user's inputs.
Stop when the user types -999999.

Ver 1. without break

```
num = int(input()) # num=0
total =0
while num != -999999:
    total+=num
    num= int(input())
print(total)
```

Ver 2. with break

idea is when num==-999999 break

```
total=0
while True:
    num= int(input())
    if num == -999999:
        break        ← only way to get out
    else:
        total+=num
print(total)
```

Q20 Input Validation

Ver 1. without break

```
hour = int(input())
while hour < 0:      ← loop while it's invalid.
    print("Invalid")
```

Ver 2. with break

```
while True:
    hour= int(input())
    if hour >= 0:    ] if valid, break
        break
```

```
    print("Invalid")
    hour = int(input())
rate = int(input())
while rate < 0:
    print("Invalid")
    rate = int(input())

salary = hour * rate
print(salary)
```

while it's Invalid.

```
    if hour >= 0:
        break
    print("Invalid")
while True:
    rate = int(input())
    if rate >= 0:
        break
    print("Invalid")
salary = hour * rate
print(salary)
```

if it's valid, break

For loop: for loop through the list of items

Syntax:

```
for var in [item1, item2, item3]:
    statement
    statement
```

① ② ③

out

* the number of items = num loop

EX

```
for x in [10, 20, 30]:
    print("Hi")
    print(x**2)
```

Output:

Hi
100
Hi
400
Hi
900

3 loops

#1
x = 10
print("Hi")
print(100)

#2
x = 20
print("Hi")
print(400)

#3
x = 30
print("Hi")
print(900)

EX User wants to enter n values for you to do the calculation.

- User will first enter n.
- Then user will enter n values or n*k values

repeat n times

EX. BMI for n people. $BMI = w/h^{**}2$ (kg) (m)

```
n = int(input("Number of people"))
for i in range(0, n, 1):
```

[0, 1, ..., n-1] = n items

n = int(input(...))

[0, 1, ..., n-1]
= n items

```
for i in range(0, n, 1):
    weight = float(input())
    height = float(input())
    print(f"BMI #{i+1} = {weight/height**2}")
```

loop n times

range() function

↳ generate a sequence of numbers

1) range(start, end, step)

end before this number!

ex: range(0, 5, 1) → [0, 1, 2, 3, 4]

range(7, 1, -3) → [7, 4]

2) range(start, end) # step = 1

ex: range(0, 5) → [0, 1, 2, 3, 4]

3) range(end) # start = 0, step = 1

ex: range(5) → [0, 1, 2, 3, 4]

Ex

```
total = 0
for n in range(4):
    total += n
print(total)
```

[0, 1, 2, 3]

0+0+1+2+3

→ $\boxed{6}$

continue statement

↳ when you encounter continue, you goto the next loop

Output: Hi
Hi
Hi
Hi

```
for x in [10, 20, 30, 40]
    print('Hi')
    continue
    print(x)
```

List Part 1

↳ ... type ... collection of items

List Part 2

List is a data type - a collection of items that we can access them by position (index)

Syntax: [item 1, item 2, item 3]

be any data type (int, float, string, ..)

Ex
```
my_list = [1, 2, "A", "B", True]
print(my_list) → [1, 2, "A", "B", True]
```

Indexing: access an item in the list by refering to its index (position)

* index starts at 0 !! *

```
index      0   1   2   3
my_list = [10, 20, 30, 40]
```

Syntax: my_list [index]

(int) a number

ex
```
print(my_list[0]) → 10
x = my_list[1] + my_list[2]
       20      +    30       = 50
print(x) → 50
```

Negative Index: the last item is -1. and you count down from the back.

```
           0   1   2   3
my_list = [10, 20, 30, 40]
           -4  -3  -2  -1
print(my_list[-1]) → 40
print(f"{my_list[2] + my_list[-3]}") → 50
          30   +   20 = 50
```

Update list: update at a specific index.

Update list : Update at a specific index.

Syntax: my_list[index] = value

update

Ex

```
list1 = [10, 20, 30, 40]
print(list1)           → [10, 20, 30, 40]
print(list1[0])        → 10
list1[0] = 5
print(list1)           → [5, 20, 30, 40]
list1[2] = list1[0] + list1[1]    5 + 20 = 25
print(list1)           → [5, 20, 25, 40]
```

0 1 2 3

* Error: index out of range *

↳ you try to access index that not in the list

```
print(list1[20])
```

try to access index 20 but it isn't in the list → Error!

---

len() function

↳ len(list) tells you how many items in the list.

```
list2 = [10, 20, 30, 40]
print(len(list2)) → 4
```

$\underbrace{\text{len(list2)}}_{4}$

```
x = len([1, 2, 5, 9, 16]) = 5
print(x) → 5
```

List Operators

- \+ : concatenate two lists together

[1, 2] + [3, 4] → [1, 2, 3, 4]

["A", 1] + ["B", 3] → ["A", 1, "B", 3]

["A",1] + ["B",3] → ["A",1,"B",3]

- * : Syntax: list * n
  ↳ repeat the list n times

[1,2] * 3 → [1,2,1,2,1,2]
3 times

Loop through list

① numbers = [10, 20, 30, 40]

```
for num in numbers:
    print(num)
```

Output: 10
20
30
40

Ex numbers = [10, 20, 30, 40] (0 1 2 3)

```
for x in [0,1]:
    print(num[x])
```

#1 x=0
print(num[0]) → 10
#2 x=1
print(num[1]) → 20

② loop through the list of indexes

```
numbers = [10, 20, 30, 40]
nlen = len(numbers) = 4
indexes = range(nlen) → [0, 1, 2, 3]
for i in indexes:
    print(numbers[i])
```

0 1 2 3 → [0, 1, 2, 3]
range(4)
= numbers of items

#1
i=0
print(numbers[0])
#2
i=1
...

Output 10 numbers[0]
20 numbers[1]
30 numbers[2]
40 numbers[3]

```
numbers = [10, 20, 30, 40]
for i in range(len(numbers)):
    print(numbers[i])
```

range(4) → [0,1,2,3]

Ex update the list

Ex Update the list

[1, 2, 3, 4] → [1, 4, 9, 16]

Update syntax!
list[index] = newvalue

```
nums = [1, 2, 3, 4]
for i in range(len(nums)):
    nums[i] = nums[i] ** 2

print(nums) → [1, 4, 9, 16]
```

Nested Loops : loop can be inside of another loop

```
for a in [1, 2, 3]:
    for b in [4, 5]:
        print(a,b)
```

loop 3 times

2 times

a = 1
b = 4
b = 5

a = 2
b = 4
b = 5

a = 3
b = 4
b = 5

Output:

```
1 4
1 5
2 4
2 5
3 4
3 5
```

```
total = 0
for i in range(0, 3):        [0, 1, 2]
    for j in [3, 5, 7]:
        if (i + j) % 2 == 0:     j
            total += (i * j)
print(total) → 15
```

j \ i	0	1	2
3	0,3 =3	1*3=3 1,3 =4	2,3 =5
5	0,5 =5	1*5=5 1,5 =6	2,5 =7
7	0,7 =7	1*7=7 1,7 =8	2,7 =9

total += 3 + 5 + 7 = $\boxed{15}$